



Universidad
Rey Juan Carlos

Procesadores De Lenguajes

[MEMORIA PRÁCTICA OBLIGATORIA]

JUAN GOMEZ REY

MARÍA ABDALLAH EL LAKKIS

CARLA GARCÍA ROMERO

Índice

Reglas Léxicas.....	2
Transformación de la gramática a LL(1).....	3
varlist.....	3
dcl.....	3
casos.....	3
sent.....	4
Casos de prueba.....	5
Casos correctos.....	5
Casos erróneos.....	12
Recuperación de Errores.....	15
Anexo: Conjuntos Directores.....	16

Reglas Léxicas

NUM_INT_CONST_B: 'b'[01]+'';

NUM_INT_CONST_O: 'o'[0-7]+'';

NUM_INT_CONST_H: 'z'[0-9A-F]+'';

Definición de **constantes numéricas enteras en diferentes bases**. Se han definido tres formatos para la representación de enteros en bases no decimales, que utilizan un prefijo identificador y delimitadores específicos.

NUM_REAL_CONST: NUM_INT_CONST '.' [0-9]+ //Punto fijo

| NUM_INT_CONST ('e'|'E') NUM_INT_CONST// Exponencial

| NUM_INT_CONST '.' [0-9]+ ('e'|'E') NUM_INT_CONST; // Mixto

Definición de **constantes numéricas reales**. Expresa números de coma flotante:

- Punto fijo: Un entero seguido de un punto y una parte fraccionaria.
- Exponencial: Un entero seguido de la letra e o E y de un exponente entero.
- Formato mixto: Combina la parte fraccionaria con la notación exponencial.

STRING_CONSTANT: '\'' (~['\n\r'] | '\\'\'')* '\''

| '"' (~["\n\r"] | '\"\'')* '"';

Definición de **constantes literales**. Las cadenas de caracteres pueden delimitarse tanto por comillas simples como por comillas dobles. Si el delimitador aparece dentro del contenido, debe duplicarse.

COMMENT: '!' ~[\n\r]* ('\n' | '\r');

Definición del formato de los **comentarios**: Son líneas de texto que comienzan con el carácter exclamación. La expresión detecta todos los caracteres hasta encontrar un salto de línea o un retorno de carro.

IDENT: [a-zA-Z]+ [a-zA-Z0-9_]*;

Definición del formato de los **identificadores**: Representan nombres de variables, funciones o estructuras. Comienza con una letra y pueden ser seguidos por cualquier combinación de letras, números o guiones bajos

Transformación de la gramática a LL(1)

Para adaptar la gramática y que sea LL(1) fue necesario transformar distintas reglas que presentaban problemas de ambigüedad o recursividad por la izquierda. A continuación se muestran algunos ejemplos relevantes del proceso seguido:

varlist

```
varlist: IDENT init | varlist ',' IDENT init;
```

Esta regla no era compatible con las gramáticas LL(1) debido a que presentaba recursividad por la izquierda. Para solucionarlo, se aplicó el algoritmo de eliminación de recursividad, introduciendo la regla auxiliar `varlist_P`.

```
varlist: IDENT init varlist_P;  
varlist_P: ',' IDENT init varlist_P | ;
```

dcl

```
dcl: defcte | defvar;
```

Dado que las reglas `defcte` y `defvar` comenzaban con el símbolo `tipo`, la intersección de los conjuntos directores de `dcl` no era vacía. La solución consistió en factorizar la regla para que comience por `tipo` (eliminandolo de las otras dos reglas) y crear una regla auxiliar que agrupe el resto de `defcte` y `defvar`.

```
dcl: tipo def_P;  
def_P: defcte | defvar;  
defcte: ',' 'PARAMETER' '::' IDENT '=' simpvalue ctelist ';';  
defvar: '::' varlist ';';
```

casos

```
casos: 'CASE' '(' etiquetas ')' sentlist casos  
      | 'CASE' 'DEFAULT' sentlist  
      | ;
```

En esta regla se identificó el problema de los conjuntos de directores no disjuntos. Se sigue la misma metodología que en las anteriores.

```
casos: 'CASE' casos_P;  
casos_P: '(' etiquetas ')' sentlist casos_PP  
        | 'DEFAULT' sentlist  
        | ;  
casos_PP: 'CASE' casos_P  
         | ;
```

sent

```
sent:
    IDENT '=' exp ';'
    | proc_call ';'
    | 'IF' '(' expcond ')' sent
    | 'IF' '(' expcond ')' 'THEN' sentlist 'ENDIF'
    | 'IF' '(' expcond ')' 'THEN' sentlist 'ELSE' sentlist 'ENDIF'
    | 'DO' 'WHILE' '(' expcond ')' sentlist 'ENDDO'
    | 'DO' IDENT '=' doval ',' doval ',' doval sentlist 'ENDDO'
    | 'SELECT' 'CASE' '(' exp ')' casos 'END' 'SELECT' ;
```

Aquí se encontraron múltiples producciones que compartían la misma cabecera. Para resolverlo, se distribuyó el contenido en varias reglas secundarias, logrando que sent solo contenga el símbolo inicial de cada una de ellas.

```
sent:
    IDENT '=' exp ';'
    | proc_call ';'
    | 'IF' '(' expcond ')' if_P
    | 'DO' do_P
    | 'SELECT' 'CASE' '(' exp ')' casos 'END' 'SELECT';

if_P: sent
    | 'THEN' sentlist if_PP;

if_PP: 'ENDIF'
    | 'ELSE' sentlist 'ENDIF';

do_P: 'WHILE' '(' expcond ')' sentlist 'ENDDO'
    | IDENT '=' i=doval ',' f=doval ',' s=doval sentlist 'ENDDO';
```

Finalmente, algunas reglas de la gramática no se pudieron modificar por falta de conocimientos. No obstante, esto no afectó al desarrollo ya que ANTLR tiene la capacidad de reconocer gramáticas de tipo LL(k). Entre las reglas que se mantuvieron intactas se encuentran: dec_f_parmalist, sentlist_P y factorcond.

Casos de prueba

Casos correctos

Caso 1 (ej1.for): Comprobación estructuras if

```
PROGRAM pruebasIf;

INTEGER :: a = 5;

INTEGER :: b = 10;

INTEGER :: c = 0;

REAL :: x = 3.5;

CHARACTER(20) :: msg = 'hola';

IF (a < b) c = 1;

IF ((a < b) .AND. (x > 1.0)) THEN

    c = c + 10;

ENDIF

IF ((a > b) .OR. (c == 200)) THEN

    c = c + 20;

ENDIF

IF (.NOT. (a == b)) THEN

    c = c + 30;

ENDIF

IF (.TRUE.) THEN

    c = c + 40;

ENDIF

IF (.FALSE.) THEN

    c = 999;

ELSE

    c = c + 50;

ENDIF

END PROGRAM pruebasIf
```

Caso 2 (ej2.for): Comprobación subrutinas

```
PROGRAM test_matematicas ;  
  
REAL, PARAMETER :: gravedad = 9.81;  
  
INTEGER :: a = 5, b = 10;  
  
REAL :: velocidad = 15.5, tiempo = 2.0, posicion_final;  
  
INTERFACE  
    SUBROUTINE Intercambiar(x, y)  
        INTEGER, INTENT(INOUT) x;  
        INTEGER, INTENT(INOUT) y;  
    END SUBROUTINE Intercambiar  
    SUBROUTINE CalcularCaida(v0, t, g, pos)  
        REAL, INTENT(IN) v0;  
        REAL, INTENT(IN) t;  
        REAL, INTENT(IN) g;  
        REAL, INTENT(OUT) pos;  
    END SUBROUTINE CalcularCaida  
END INTERFACE  
  
IF (a /= b) THEN  
    CALL Intercambiar(a, b);  
ENDIF  
  
CALL CalcularCaida(velocidad, tiempo, gravedad, posicion_final);  
END PROGRAM test_matematicas  
  
SUBROUTINE Intercambiar(x, y)  
    INTEGER, INTENT(INOUT) x;  
    INTEGER, INTENT(INOUT) y;  
    INTEGER :: aux;  
    aux = x;  
    x = y;  
    y = aux;  
END SUBROUTINE Intercambiar
```

```
SUBROUTINE CalcularCaida(v0, t, g, pos)

  REAL, INTENT(IN) v0;

  REAL, INTENT(IN) t;

  REAL, INTENT(IN) g;

  REAL, INTENT(OUT) pos;

  pos = (v0 * t) + (0.5 * g * (t * t));

END SUBROUTINE CalcularCaida
```


Caso 3 (ej3.for): Comprobación estructuras do y select

```
PROGRAM test_bucles ;

INTEGER, PARAMETER :: limite = 10, inicio = 1, paso = 1;
INTEGER :: iterador, suma_total = 0;
INTEGER :: opcion = 2;
REAL :: calculo_extra;
CHARACTER (10):: var7, var8;
CHARACTER :: var4, var5 = "-", var6;

INTERFACE
    SUBROUTINE EvaluarOpcion(op, res)
        INTEGER, INTENT(IN) op;
        REAL, INTENT(OUT) res;
    END SUBROUTINE EvaluarOpcion
    FUNCTION fun1 ( a, b )
        INTEGER :: fun1;
        INTEGER, INTENT (IN) a;
        REAL, INTENT (IN) b;
    END FUNCTION fun1
END INTERFACE

DO iterador = inicio, limite, paso
    suma_total = suma_total + iterador;
ENDDO

IF (suma_total > 50) THEN
    calculo_extra = 1.0;
ELSE
    IF (suma_total == 50) THEN
        calculo_extra = 0.5;
    ELSE
        calculo_extra = 0.0;
    ENDIF
ENDIF

ENDIF
```

```
DO WHILE (opcion < 5)
    SELECT CASE (opcion)
        CASE (1)
            calculo_extra = calculo_extra + 10.0;
        CASE (2, 3)
            calculo_extra = calculo_extra + 20.0;
        CASE DEFAULT
            calculo_extra = calculo_extra - 1.0;
    END SELECT

    opcion = opcion + 1;
ENDDO

CALL EvaluarOpcion(opcion, calculo_extra);

END PROGRAM test_bucles

SUBROUTINE EvaluarOpcion(op, res)
    INTEGER, INTENT(IN) op;
    REAL, INTENT(OUT) res;

    res = 0.0;
END SUBROUTINE EvaluarOpcion

FUNCTION fun1 ( a, b )
    INTEGER :: fun1;
    INTEGER, INTENT (IN) a;
    REAL, INTENT (IN) b;
    CALL proc2;
    fun1 = a * 2;
END FUNCTION fun1
```

Caso 4 (ej4.for): Comprobación completa de expresiones y funciones

```
PROGRAM test_final;

INTEGER, PARAMETER :: bin = b'1010', oct = o'740', hex = z'A34';

INTEGER :: a = 2, b = 3, c = 0;
INTEGER :: resultado;
REAL :: x = 2.5, y = 4.0, z;
CHARACTER(30) :: texto = "hola ""mundo""";
CHARACTER(20) :: texto2 = 'It's ok';

INTERFACE

    FUNCTION Multiplicar(valor1, valor2)
        REAL :: Multiplicar;
        INTEGER, INTENT(IN) valor1;
        REAL, INTENT(IN) valor2;
    END FUNCTION Multiplicar

    FUNCTION Sumar(v1, v2)
        INTEGER :: Sumar;
        INTEGER, INTENT(IN) v1;
        INTEGER, INTENT(IN) v2;
    END FUNCTION Sumar

END INTERFACE

c = a + b * 2;
z = Multiplicar(c, x);
resultado = Sumar(a, b);

IF ((a < b) .AND. (.NOT. (c == 0))) THEN
    DO WHILE ((a < 10) .AND. (b < 20))
        SELECT CASE (a)
```

```
        CASE (1)
            c = c + 1;
        CASE (2,3)
            c = c + 2;
        CASE (4)
            c = c + 4;
        CASE DEFAULT
            c = -1;
        END SELECT
        a = a + 1;
    ENDDO
ELSE
    c = 999;
ENDIF

END PROGRAM test_final

FUNCTION Multiplicar(valor1, valor2)
    REAL :: Multiplicar;
    INTEGER, INTENT(IN) valor1;
    REAL, INTENT(IN) valor2;
    REAL :: temp;
    temp = (valor1 * valor2) + 3.0;
    Multiplicar = temp;
END FUNCTION Multiplicar

FUNCTION Sumar(v1, v2)
    INTEGER :: Sumar;
    INTEGER, INTENT(IN) v1;
    INTEGER, INTENT(IN) v2;
    INTEGER :: aux;
    aux = v1 + v2;
    Sumar = aux;
END FUNCTION Sumar
```

Casos erróneos

Caso 1 (ej5.for):

Este caso contiene una función suma cuya declaración de tipo de retorno está asociada al identificador resta. El procesador detecta el incumplimiento de la comprobación semántica que exige que el nombre de la función coincida con el identificador asociado al tipo devuelto.

```
PROGRAM prueba;  
  
  INTERFACE  
  
    FUNCTION suma(a,b) INTEGER :: resta;  
  
      INTEGER, INTENT(IN) a;  
  
      INTEGER, INTENT(IN) b;  
  
    END FUNCTION suma  
  
  END INTERFACE  
  
END PROGRAM prueba
```

Al detectarse el error semántico, el procesador, además, no puede continuar parseando correctamente la zona de sentencias, lo que genera un error sintáctico secundario. No se genera ninguna salida en el fichero .c.

```
Error Semántico: En la FUNCTION 'suma', el tipo de retorno está asociado a un  
identificador distinto ('resta').  
  
Error Sintáctico en línea 10, columna 0.  
Se esperaba {'IF', 'DO', 'SELECT', 'CALL', IDENT} y se encontró 'END'.  
Posible causa: Se esperaba un identificador.
```

Caso 2 (ej6.for):

Este caso define una función mult correctamente en la cabecera INTERFACE, pero su implementación aparece dentro del cuerpo del programa principal en lugar de en la zona subproglis, que es después de END PROGRAM.

```
PROGRAM prueba;  
  
  INTERFACE  
  
    FUNCTION mult(a,b) INTEGER :: mult;  
  
      INTEGER, INTENT(IN) a;  
  
      INTEGER, INTENT(IN) b;  
  
    END FUNCTION mult  
  
  END INTERFACE  
  
  FUNCTION mult(a,b) INTEGER :: mult;  
  
    INTEGER, INTENT(IN) a;  
  
    INTEGER, INTENT(IN) b;  
  
    x = a * b;  
  
  END FUNCTION mult  
  
END PROGRAM prueba
```

El parser no reconoce la palabra FUNCTION como una sentencia válida en ese contexto, generando varios errores sintácticos en cascada. No se genera ninguna salida en el fichero .c.

```
Error Sintáctico en línea 10, columna 4.  
Se esperaba símbolos válidos y se encontró 'FUNCTION'.  
Posible causa: Construcción sintáctica no reconocida.  
  
Error Sintáctico en línea 10, columna 17.  
Se esperaba símbolos válidos y se encontró '('.  
Posible causa: Construcción sintáctica no reconocida.  
  
Error Sintáctico en línea 10, columna 19.  
Se esperaba {'(', ')', '+', '-', '*', '/'} y se encontró ','.  
Posible causa: Falta el paréntesis de cierre.
```

Caso 3 (ej7.for):

Este caso contiene un END SELECT que no corresponde a ningún SELECT CASE abierto.

```
PROGRAM errorSintactico;  
INTEGER :: x = 3, y = 2;  
IF (y < x) x = x + 1;  
END SELECT  
END PROGRAM errorSintactico
```

El parser no lo encuentra en un punto donde espera una sentencia o el fin del programa, y como END SELECT no es una sentencia válida, genera un error. No se genera ninguna salida en el fichero .c.

```
Error Sintáctico en línea 6, columna 4.  
Se esperaba {';', '+', '-', '*', '/'} y se encontró 'SELECT'.  
Posible causa: Falta el punto y coma al final de la sentencia.
```

Caso 4 (ej8.for):

Este caso contiene un bucle DO WHILE que nunca se cierra con ENDDO.

```
PROGRAM errorEnddo;  
INTEGER :: i = 0;  
DO WHILE (i < 5)  
    i = i + 1;  
END PROGRAM errorEnddo
```

El parser llega al END PROGRAM esperando más sentencias dentro del cuerpo del bucle o el ENDDO correspondiente, y al encontrar END en su lugar genera un error sintáctico.

```
Error Sintáctico en línea 7, columna 0.  
Se esperaba símbolos válidos y se encontró 'END'.  
Posible causa: Construcción sintáctica no reconocida.
```

Recuperación de Errores

Tras la lectura del libro propuesto por los profesores, se determinó que, para el bloque de **recuperación de errores** (calificación de notable), no es requerida ninguna implementación adicional, dado que ANTLR integra este mecanismo de forma nativa. El analizador no detiene su ejecución ante un fallo sintáctico, sino que continúa procesando la entrada.

Por el contrario, para optar a la calificación de sobresaliente, sí es indispensable desarrollar una clase específica que capture y traduzca dichos errores. Esta implementación se ha llevado a cabo mediante un `ErrorListener` personalizado.

Esta clase obtiene, a partir del parser, los tokens esperados en cada punto de error. Con esta información, el sistema genera mensajes de diagnóstico en español que detallan la línea, la columna, los símbolos esperados y el símbolo erróneo encontrado.

Además, tras la identificación de los tokens esperados, la clase genera un mensaje complementario que aporta una explicación más detallada sobre la posible causa del error.

Anexo: Conjuntos Directores

- $\text{DIR}(\text{prg} : \text{'PROGRAM' IDENT ';' dcllist cabecera sentlist 'END' 'PROGRAM' IDENT subproglis}) = \{ \text{'PROGRAM'} \}$
- $\text{DIR}(\text{dcllist} : \text{lambda}) = \{ \text{'INTERFACE', 'CALL ', ' IDENT ', ' IF ', ' DO ', ' SELECT ' } // \text{DEPENDE DE SIG dcllist}$
- $\text{DIR}(\text{dcllist} : \text{dcl dcllist}) = \{ \text{'INTEGER', 'REAL', 'CHARACTER'} \}$
- $\text{DIR}(\text{cabecera} : \text{lambda}) = \{ \text{'CALL ', ' IDENT ', ' IF ', ' DO ', ' SELECT '}$
- $\text{DIR}(\text{cabecera} : \text{'INTERFACE' cablist 'END' 'INTERFACE'}) = \{ \text{'INTERFACE'} \}$
- $\text{DIR}(\text{cablist} : \text{decproc decsubprog}) = \{ \text{'SUBROUTINE'} \}$
- $\text{DIR}(\text{cablist} : \text{decfun decsubprog}) = \{ \text{'FUNCTION'} \}$
- $\text{DIR}(\text{decsubprog} : \text{lambda}) = \{ \text{'END'} \}$
- $\text{DIR}(\text{decsubprog} : \text{decproc decsubprog}) = \{ \text{'SUBROUTINE'} \}$
- $\text{DIR}(\text{decsubprog} : \text{decfun decsubprog}) = \{ \text{'FUNCTION'} \}$
- $\text{DIR}(\text{sentlist} : \text{sent sentlist_P}) = \{ \text{'CALL ', ' IDENT ', ' IF ', ' DO ', ' SELECT '}$
- $\text{DIR}(\text{sentlist_P} : \text{sent sentlist_P}) = \{ \text{'CALL ', ' IDENT ', ' IF ', ' DO ', ' SELECT '}$
- $\text{DIR}(\text{sentlist_P} : \text{lambda}) = \{ \text{'END', 'IDENT'}$
- $\text{DIR}(\text{dcl} : \text{tipo def_P}) = \{ \text{'INTEGER', 'REAL', 'CHARACTER'} \}$
- $\text{DIR}(\text{def_P} : \text{defcte}) = \{ \text{' '} \}$
- $\text{DIR}(\text{def_P} : \text{defvar}) = \{ \text{'::'} \}$
- $\text{DIR}(\text{defcte} : \text{' ' 'PARAMETER' '::' IDENT '=' simpvalue ctelist ';' }) = \{ \text{' '} \}$
- $\text{DIR}(\text{ctelist} : \text{' ' IDENT '=' simpvalue ctelist; }) = \{ \text{' '} \}$
- $\text{DIR}(\text{ctelist} : \text{lambda}) = \{ \text{';' } \}$

- $\text{DIR}(\text{simpvalue} : \text{NUM_INT_CONST}) = \{ \text{NUM_INT_CONST} \}$
- $\text{DIR}(\text{simpvalue} : \text{NUM_REAL_CONST}) = \{ \text{NUM_REAL_CONST} \}$
- $\text{DIR}(\text{simpvalue} : \text{STRING_CONSTANT}) = \{ \text{STRING_CONSTANT} \}$
- $\text{DIR}(\text{simpvalue} : \text{NUM_INT_CONST_O}) = \{ \text{NUM_INT_CONST_O} \}$
- $\text{DIR}(\text{simpvalue} : \text{NUM_INT_CONST_H}) = \{ \text{NUM_INT_CONST_H} \}$
- $\text{DIR}(\text{simpvalue} : \text{NUM_INT_CONST_B}) = \{ \text{NUM_INT_CONST_B} \}$

- $\text{DIR}(\text{charlength} : \text{lambda}) = \text{SIG}(\text{charlength}) = \{ ', '::' \}$
- $\text{DIR}(\text{charlength} : '(' \text{ NUM_INT_CONST} ')') = \{ '(' \}$

- $\text{DIR}(\text{varlist} : \text{IDENT init varlist_P}) = \{ \text{IDENT} \}$

- $\text{DIR}(\text{varlist_P} : \text{lambda}) = \text{SIG}(\text{varlist_P}) = \{ '; ' \}$
- $\text{DIR}(\text{varlist_P} : ', ' \text{ IDENT init varlist_P}) = \{ ', ' \}$

- $\text{DIR}(\text{init} : \text{lambda}) = \text{SIG}(\text{init}) = \{ ', ', '; ' \}$
- $\text{DIR}(\text{init} : '=' \text{ simpvalue}) = \{ '=' \}$

- $\text{DIR}(\text{decproc} : \text{'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist 'END' 'SUBROUTINE' IDENT}) = \{ \text{'SUBROUTINE'} \}$

- $\text{DIR}(\text{formal_paramlist} : \text{lambda}) = \text{SIG}(\text{formal_paramlist}) = \{ \text{'END'} \}$
- $\text{DIR}(\text{formal_paramlist} : '(' \text{ nomparamlist} ')') = \{ '(' \}$

- $\text{DIR}(\text{nomparamlist} : \text{IDENT nomparamlist_P}) = \{ \text{IDENT} \}$

- $\text{DIR}(\text{nomparamlist_P} : \text{lambda}) = \{ \text{IDENT} \}$
- $\text{DIR}(\text{nomparamlist_P} : ', ' \text{ nomparamlist}) = \{ ', ' \}$

- $\text{DIR}(\text{dec_s_paramlist} : \text{lambda}) = \text{SIG}(\text{dec_s_paramlist}) = \{ \text{'END'}, \text{'INTEGER'}, \text{'REAL'}, \text{'CHARACTER'}, \text{IDENT}, \text{'CALL'} \}$
- $\text{DIR}(\text{dec_s_paramlist} : \text{tipo } ', ' \text{'INTENT'} (' \text{tipoparam} ') \text{IDENT } '; ' \text{dec_s_paramlist}) = \text{CAB}(\text{tipo}) = \{ \text{'INTEGER'}, \text{'REAL'}, \text{'CHARACTER'} \}$
- $\text{DIR}(\text{tipoparam} : \text{'IN'}) = \{ \text{'IN'} \}$
- $\text{DIR}(\text{tipoparam} : \text{'OUT'}) = \{ \text{'OUT'} \}$
- $\text{DIR}(\text{tipoparam} : \text{'INOUT'}) = \{ \text{'INOUT'} \}$
- $\text{DIR}(\text{decfun} : \text{'FUNCTION'} \text{IDENT } '(' \text{nomparamlist } ') ' \text{tipo } ':: ' \text{IDENT } '; ' \text{dec_f_paramlist } \text{'END'} \text{'FUNCTION'} \text{IDENT}) = \{ \text{'FUNCTION'} \}$
- $\text{DIR}(\text{dec_f_paramlist} : \text{lambda}) = \text{SIG}(\text{dec_f_paramlist}) = \{ \text{'END'}, \text{'INTEGER'}, \text{'REAL'}, \text{'CHARACTER'}, \text{IDENT}, \text{'CALL'} \}$
- $\text{DIR}(\text{dec_f_paramlist} : \text{tipo } ', ' \text{'INTENT'} (' \text{'IN'} ') \text{IDENT } '; ' \text{dec_f_paramlist}) = \text{CAB}(\text{tipo}) = \{ \text{'INTEGER'}, \text{'REAL'}, \text{'CHARACTER'} \}$
- $\text{DIR}(\text{exp} : \text{factor } \text{exp_p}) = \text{CAB}(\text{factor}) = \{ \text{NUM_INT_CONST}, \text{NUM_REAL_CONST}, \text{STRING_CONSTANT}, '(', \text{IDENT} \}$
- $\text{DIR}(\text{exp_p} : \text{op } \text{exp } \text{exp_p}) = \text{CAB}(\text{op}) = \{ \text{'+'}, \text{'-'}, \text{'*'}, \text{'/'} \}$
- $\text{DIR}(\text{exp_p} : \text{lambda}) = \text{SIG}(\text{exp_p}) = \{ ')', ',', ';' \}$
- $\text{DIR}(\text{op} : \text{'+'}) = \{ \text{'+'} \}$
- $\text{DIR}(\text{op} : \text{'-'}) = \{ \text{'-'} \}$
- $\text{DIR}(\text{op} : \text{'*'}) = \{ \text{'*'} \}$
- $\text{DIR}(\text{op} : \text{'/'}) = \{ \text{'/'} \}$
- $\text{DIR}(\text{factor} : \text{simpvalue}) = \text{CAB}(\text{simpvalue}) = \{ \text{NUM_INT_CONST}, \text{NUM_REAL_CONST}, \text{STRING_CONSTANT}, \text{NUM_INT_CONST_B}, \text{NUM_INT_CONST_H}, \text{NUM_INT_CONST_O} \}$
- $\text{DIR}(\text{factor} : '(' \text{exp} ')') = \{ '(' \}$
- $\text{DIR}(\text{factor} : \text{IDENT } \text{factor_P}) = \{ \text{IDENT} \}$

-
- DIR(factor_P : '(' exp explist ')') = { '(' }
 - DIR(factor_P : lambda) = { '+', '-', '*', '/', ',', ';', '}' }

 - DIR(explist : ',' exp explist) = { ',' }
 - DIR(explist : lambda) = SIG(explist) = { '}' }

 - DIR(subpparamlist : '(' exp explist ')') = { '(' }
 - DIR(subpparamlist : lambda) = SIG(subpparamlist) = { ';' }

 - DIR(subproglis : codproc subproglis) = CAB(codproc) = { 'SUBROUTINE' }
 - DIR(subproglis : codfun subproglis) = CAB(codfun) = { 'FUNCTION' }
 - DIR(subproglis : lambda) = SIG(subproglis) = { '\$' }

 - DIR(codproc : 'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist dllist sentlist 'END' 'SUBROUTINE' IDENT) = { 'SUBROUTINE' }

 - DIR(codfun : codfun: 'FUNCTION' IDENT '(' nomparamlist ')' tipo '::' IDENT ';' dec_f_paramlist dllist sentlist IDENT '=' exp ';' 'END' 'FUNCTION' IDENT) = { 'FUNCTION' }

 - DIR(expcond : factorcond expcond_P) = CAB(factorcond) = { NUM_INT_CONST, NUM_INT_CONST_B, NUM_INT_CONST_H, NUM_INT_CONST_O, NUM_REAL_CONST, STRING_CONSTANT, '(', IDENT, '.NOT.', '.TRUE.', '.FALSE.' }

 - DIR(expcond_P : oplog factorcond expcond_P) = { '.OR.', '.AND.', '.EQV.', '.NEQV.' }
 - DIR(expcond_P : lambda) = SIG(expcond_P) = SIG(expcond) = { '}' }

 - DIR(oplog : '.OR.') = { '.OR.' }
 - DIR(oplog : '.AND.') = { '.AND.' }
 - DIR(oplog : '.EQV.') = { '.EQV.' }
 - DIR(oplog : '.NEQV.') = { '.NEQV.' }

-
- DIR(factorcond : exp opcomp exp) = CAB(exp) = { NUM_INT_CONST, NUM_INT_CONST_B, NUM_INT_CONST_H, NUM_INT_CONST_O, NUM_REAL_CONST, STRING_CONSTANT, '(', IDENT }
 - DIR(factorcond : '(' expcond ')') = { '(' }
 - DIR(factorcond : '.NOT.' factorcond) = { '.NOT.' }
 - DIR(factorcond : '.TRUE.') = { '.TRUE.' }
 - DIR(factorcond : '.FALSE.') = { '.FALSE.' }

 - DIR(opcomp : '<') = { '<' }
 - DIR(opcomp : '>') = { '>' }
 - DIR(opcomp : '<=') = { '<=' }
 - DIR(opcomp : '>=') = { '>=' }
 - DIR(opcomp : '==') = { '==' }
 - DIR(opcomp : '/=') = { '/=' }

 - DIR(sent : IDENT '=' exp ';') = { IDENT }
 - DIR(sent : proc_call ';') = CAB(proc_call) = { 'CALL' }
 - DIR(sent : 'IF' '(' expcond ')' if_P) = { 'IF' }
 - DIR(sent : 'DO' do_P) = { 'DO' }
 - DIR(sent : 'SELECT' 'CASE' '(' expcond ')' casos 'END' 'SELECT') = { 'SELECT' }

 - DIR(if_P : sent) = CAB(sent) = { 'CALL ', ' IDENT ', ' IF ', ' DO ', ' SELECT ' }
 - DIR(if_P : 'THEN' sentlist if_PP) = { ' THEN ' }

 - DIR(if_PP : 'ENDIF') = { 'ENDIF' }
 - DIR(if_PP : 'ELSE' sentlist 'ENDIF') = { 'ELSE' }

 - DIR(do_P : 'WHILE' '(' expcond ')' sentlist 'ENDDO') = { 'WHILE' }
 - DIR(do_P : IDENT '=' doval ', ' doval ', ' doval sentlist 'ENDDO') = { IDENT }

 - DIR(doval : 'NUM_INT_CONST') = { 'NUM_INT_CONST' }
 - DIR(doval : 'IDENT') = { 'IDENT' }

-
- $\text{DIR}(\text{etiquetas: simpvalue etiquetas_P}) = \text{CAB}(\text{simpvalue}) = \{ \text{NUM_INT_CONST}, \text{NUM_INT_CONST_B}, \text{NUM_INT_CONST_H}, \text{NUM_INT_CONST_O}, \text{NUM_REAL_CONST}, \text{STRING_CONSTANT} \}$
 - $\text{DIR}(\text{etiquetas : ':' simpvalue}) = \{ ':' \}$

 - $\text{DIR}(\text{etiquetas_P: listaetiquetas}) = \text{CAB}(\text{listaetiquetas}) = \{ ', ' \}$
 - $\text{DIR}(\text{etiquetas_P: ':' etiquetas_PP}) = \{ ':' \}$
 - $\text{DIR}(\text{etiquetas_P: lambda}) = \text{SIG}(\text{etiquetas_P}) = \{ ' ' \}$

 - $\text{DIR}(\text{etiquetas_PP: simpvalue}) = \{ \text{NUM_INT_CONST}, \text{NUM_INT_CONST_B}, \text{NUM_INT_CONST_H}, \text{NUM_INT_CONST_O}, \text{NUM_REAL_CONST}, \text{STRING_CONSTANT} \}$
 - $\text{DIR}(\text{etiquetas_PP: lambda}) = \text{SIG}(\text{etiquetas_PP}) = \{ ' ' \}$

 - $\text{DIR}(\text{casos : 'CASE' casos_P}) = \{ 'CASE' \}$

 - $\text{DIR}(\text{casos_P : '(' etiquetas ')' sentlist casos}) = \{ '(' \}$
 - $\text{DIR}(\text{casos_P : 'DEFAULT' sentlist}) = \{ 'DEFAULT' \}$
 - $\text{DIR}(\text{casos_P : lambda}) = \text{SIG}(\text{casos_P}) = \{ 'END' \}$

 - $\text{DIR}(\text{casos_PP : 'CASE' casos_P}) = \{ 'CASE' \}$
 - $\text{DIR}(\text{casos_PP : lambda}) = \text{SIG}(\text{casos_PP}) = \{ 'END' \}$

 - $\text{DIR}(\text{listaetiquetas : ', ' simpvalue listaetiquetas}) = \{ ', ' \}$
 - $\text{DIR}(\text{listaetiquetas : lambda}) = \{ ' ' \}$